CS2109S Introduction to AI. and ML

AY24/25 Sem 2. github.com/gerteck

• **Search Algorithms** (Uninformed Search (BFS/DFS), Local Search (Hill Climbing), Informed Search (A*), Adversarial Search (Minimax))
• **"Classical" ML** (Decision Trees, Linear/Logistic Regression, Kernels and Support Vector Machines, "Classical" Unsupervised Learning)
• **"Modern" ML** (Neural Networks, Deep Learning, Sequential Data)

1. Introduction to AI

Intelligent Agent and Environments

An agent perceives environment through sensors, act through actuators.
Rational: choose actions to maximize expected performance.

- **Percept**: content agent's sensors are perceiving. **Percept Sequence** complete history of things agent has perceived.
- **PEAS**: Performance Measure (definition of success), Environment (world agent operates in), Actuators (actions agent can perform), Sensors (Inputs agent gets from environment)
  (E.g. Chatbot: Sensor as text input/chat history, Actuators as output, Environment as chat interface, plugins, Perf. measure as correctness safety.)
- **Exploration vs. Exploitation**: Exploration: learn more about world. Exploitation: Maximize gain based on current knowledge.

Properties of Task Environments

Observable: Fully vs. Partially observable.
Deterministic vs. Stochastic: Next state determined by current state and action. If environment also dependent on actions of other agents, also *strategic*, unless other agents are predictable.
Episodic vs. Sequential: Single-step vs. multi-step tasks. (Next episode does/does not depend on actions taken in previous episode.)
Static vs. Dynamic: Environment changes while deliberating. - semi-dynamic (env no change, only change agent performance score)
Discrete vs. Continuous: Finite (distinct, clearly defined) vs. infinite (states/percepts)/actions.
Single-agent vs. Multi-agent: Operating alone or with others.

Common Agent Structures

Simple Reflex Agent: Condition-action rules (e.g., "If up empty: up"). Based on current percept, ignore percept history.
Goal-Based Agent: Acts to achieve goals. Tracks state of world, goals.
Utility-Based Agent: Maximizes utility (e.g., rewards). Use utility function (perf. measure) to assign scores.
Learning Agent: Adapts and improves over time, using performance element, critic and problem generator.

2. Problem Solving by Searching

Search Problem

Search problem defined where goal is to find state/(path to state) from a set of possible states.
State Space. Set of possible states environment can be.
Representation Invariant: Abstract states hv. mapped concrete states. **Initial State** where agent starts, **Goal State(s)**: can be > 1, use goal test.
Actions. Given state s, finite set of actions that can be executed.
Transition Model. Describes what each action does. Returns state resulting from action a in state s.
Action Cost Function: Numeric cost of applying action to reach next state.
Search Algorithm: Takes search problem as input, returns a solution / failure. Solution could be sequence of actions, or final state. Optimal solution is one with least cost.

Search Algorithms

Search algorithms explore the state space to find solution. Key operations:
Frontier: Set of nodes to explore next. *Explored Set*: Nodes already visited. *Node*: Contains state, parent node, action, path cost, and depth.

Performance/Evaluation
Completeness: Finds solution if exists / report failure for none.
Optimality (Cost) : If outputs solution, it is lowest path cost of all solutions. Algorithm can be both optimal and incomplete.
Time, Space complexity: How long, how much memory needed.

Uninformed Search Algorithms

No information to guide search, aka blind search, no clue how good state is.

Breadth-First Search (BFS)

- **Frontier**: Queue (FIFO). Explore all nodes at curr. depth, before deeper.
- **Complete**: if branching factor b finite. **Optimal**: if step cost uniform.
- **Complexity**: Time: $O(b^d)$, d is depth of shallowest sln. Space: $O(b^d)$.
- *Optimize*: track visited set, early goal test (check node soln once generated).

Uniform-Cost Search (UCS) (Dijkstra's SSSP 'equiv')

- **Frontier**: Priority queue by min total path cost. Expand least-cost first.
- No-visited-mem ('tree-search', as if tree). Visited-mem ('graph-search').
- Even if goal state in frontier, sorted, first g-state selected must be optimal.
- **Complete**: if step costs are positive. **Optimal**: if step costs are positive.
- **Complexity**: Time: $O(b^{C^*/\epsilon})$, C^* is the optimal cost, ϵ is minimum step cost. Space: $O(b^{C^*/\epsilon})$. (w.r.t 'tier' of optimal solution)

Depth-First Search (DFS)

- **Frontier**: Stack (LIFO). Explores as deep as possible before backtrack.
- **Incomplete**: if search tree has infinite depth or loops.
  Non-optimal: No, may find suboptimal solutions.
- **Complexity**: Time: $O(b^m)$, where m is max depth. Space: $O(bm)$.

Depth-Limited Search (DLS)

- Limit search depth to l. Backtrack when limit hit. Good if opt. d known.
- **Incomplete**: if sln exists beyond depth l. **Non-optimal**: if used w. DFS.
- **Complexity**: Time: $O(b^l)$. Space: $O(bl)$.

Iterative Deepening Search (IDS)

- Repeatedly applies DLS with increasing depth limits. Try all possible depth limits from 0 till sln found or failure value returned from DLS. Combines benefits of BFS and DFS.
- **Complete**: If branching factor b finite. **Optimal**: if step costs uniform.
- **Complexity**: Time : $O(b^d)$. Space: $O(bd)$.

Informed Search Algorithms

Guide search with extra info, aka heuristic function on how close to goal.
Heuristic $h(n)$ estimates cost to reach goal from node n. $h^*(n)$: true cost.

- **Admissible**: Never overestimates cost ($h(n) \leq h^*(n)$). (e.g. $h(n) = 0$)
- **Consistent**: if for every node n, every successor n' of n generated by any action a: $h(n) \leq c(n, a, n') + h(n')$ and $h(G) = 0$. Δ Inequality.
- *Relaxed Problem*: Problem w. fewer restrictions. Cost of optimal solution to relaxed problem is admissible heuristic for original problem. (e.g. straight line distance for cities distances).
- *Dominance*: If $h_1(n) \geq h_2(n)$ for all n, h_1 dominates h_2. Dominant heuristics better for search if admissible. (e.g. straight line distance, vs trivial heuristic $h = 0$.)

Best-First Search

- **Frontier**: Priority queue ordered by evaluation function $f(n)$. Expands most promising node based on just $f(n)$ (no past cost).
- Not complete, not cost-optimal.

A* Search

- **Frontier**: Priority queue. (Best-First Search considering path cost).
- Use evaluation $f(n) = g(n) + h(n)$. $g(n)$ is cost to reach n from start. $h(n)$ is estimated cost to reach goal from n.
- **Complete**: if $h(n)$ is admissible, state space finite, step costs positive. **Optimal** if $h(n)$ is admissible.
- **Complexity**: Time: Exponential but improved by good heuristics. Space: Exponential. (Both $O(b^d)$ for poor heuristic.)

If $h(n)$ admissible, A* search (w/o visited memory) is optimal. If $h(n)$ consistent, A* search (w. visited memory) is optimal.

- A consistent heurisitic $\implies$ admissible (T2.QnC.1). Converse not true.
- (Trivial $h = 0$ consistent, admissible)
- Triangle inequality: Sum of lengths of any two sides $\geq$ length of 3^{rd} side.

3. Local Search

For (NP-hard) problems, traverse state space in systematic manner not option, is intractable (unsolvable in time).

Local Search: Explore state space considering only locally-reachable states ('search space'). Start random position, iterative move to neighboring states via perturbation or construction. Solution is final state, not path taken.
(E.g. N-Queens Problem, minimize function value)

- Typically incomplete and suboptimal.
- *Any-time property*: Longer runtime yields better solutions. Suitable for obtaining "good enough" solutions for difficult problems.
- **Types**: *Perturbative* Search (search space is complete solutions, modify components of current solution). *Constructive* Search (partial candidate solutions, extend by adding components).

Problem Formulation in Local Search:
States: Represent configurations/candidate sln within search space, but may not directly map to actual state. *Goal Test* (Optional) checks if current state is desired solution. *Successor Function*: Generates neighboring states by applying small modifications.
Evaluation/Objective Function: assess quality of state, max/minimize based on problem. (E.g. N-Queen Maximize "safe" queens, min. y value.)

Hill-Climbing Algorithm

Find local maxima by travelling to neighbouring states with highest value, terminate when no neighbour has higher value. One objective function is to negate heuristic function.

```
current_state = initial_state
while True:
    best_successor = highest-valued successor of curr_state
    if eval(best_successor) <= eval(current_state):
        return current_state
    current_state = best_successor
```

Greedy local search: gets stuck at local maxima, ridges, plateaus, shoulders (same values flat area). Some solutions include limited sideways move, stochastic (probability based on steepnes of move), first-choice, random-restart. Simulated Annealing - allow occasional 'bad moves' to escape local optima.

4. Adversarial Search and Games

'Classicial' Adversarial: Agents/Player compete against each other. Look at deterministic, two player, turn-taking, perfect information, zero-sum games. Perfect information: fully observable. Zero sum: no "win-win".

- *States*: Configurations of game, initial state, terminal state (win/lose/draw).
- *Actions*: Possible moves by a player.
- *Transition Model*: Defines how actions change the state.
- *Utility Function*: Assigns a value to terminal states from perspective of the agent. (E.g. Win: $+1$, Draw: 0, Loss: -1)

Minimax Algorithm

Determine optimal move for Player A (MAX) assuming Player B (MIN) plays optimally. Work out minimax value of each state in tree.

- **Complexity**: Time: Exponential w.r.t. max depth ($O(b^m)$). Space: Poly.
- **Complete**: If tree is finite. **Optimal** if against optimal opponent.

It is a recursive algo, backs up minimax values as recursion unwinds. Thus, similar to DFS, time c. $O(b^m)$, space c. $O(bm)$.
Note: minimax not 'optimal' against non-optimal MIN player, as non optimal player could choose higher value in unchosen path (which had lower min value). But guarantees it never lower utility than of optimal opponent.

Alpha-Beta Pruning (Minimax)

Reduces number of nodes (computational overhead) evaluated in game tree without affecting final decision. Minimax no need explore full tree. Good move ordering improves effectivness of pruning.

- α: MAX's highest guaranteed so far. MAX updates α.
- β: MIN's lowest guarantee so far. MIN updates β.
- At MAX node, prune (stop) if $\alpha \geq \beta$. At MIN node, prune (stop) if $\beta \leq \alpha$.
- Values of α, β passed down tree as explore nodes.
  When backtracking, values passed up to update α, β if applicable.

Good move ordering improves effectivness of pruning.

Evaluation Function to Cutoff (Minimax)

Fully exploring game tree with Minimax may be computationally infeasible.

- Apply **cutoff strategy** to halt the search at predefined maximum depth.
- Use **heuristic function** to estimate the utility of mid-game (non-terminal) states. Heuristic must be some value between win and loss, and computationally efficient. Admissibility, consistency properties do not apply. Can use domain-specific features or some labeled dataset to train heuristics.

5. Machine Learning

Computers with ability to learn without explicit programming, through algorithms that learn from, and make predictions based on data. Goal is for performance to improve over time by identifying patterns in the data.

Problem Types

- **Classification**: Output category value in finite set. (Predict discrete label)
- **Regression**: Output is number. (Predict continuous num. value)

Types of Machine Learning (Feedback difference)

1. **Supervised Learning**: Learns from labeled data (input-output pairs) to map inputs to outputs. (E.g. classify pic banana or apple)
2. **Unsupervised Learning**: Learns from unlabeled data to find patterns or structure. (E.g. clustering, group based on features).
3. **Semi-Supervised Learning**: Use labeled, unlabeled data for learning.
4. **Reinforcement Learning**: Learns to make decisions by interacting with an environment, (rewards and penalties).

Supervised Learning (Decision Trees, Linear/Logistic Regression, SVMs)

- **Training Phase**: Map inputs to output, using train data. Result: *model / hypothesis*.
- **Testing Phase**: Model's performance on test data to evaluate generalization.
- Supervised learning: dataset represented as set of pairs $(x^{(i)}, y^{(i)})$:
  - $x^{(i)} \in \mathbb{R}^d$: Input vector of i-th data point with d features.
  - $y^{(i)}$: Label (target) for the i-th data point. ($y^{(i)} \in \mathbb{R}$ or $\in \{1, \ldots, C\}$)
  - Dataset D with n examples: $D = \{(x^{(1)}, y^{(1)}) \ldots, (x^{(n)}, y^{(n)})\}$.

Goal: to find hypothesis $h(x)$ that approximates the true function f^*, assuming an unknown true relationship $y = f^*(x) + \epsilon$, ϵ noise error term.

- **Hypothesis class** $\mathcal{H}$: set of all possible learnable models or functions. Each $h \in \mathcal{H}$ is a hypothesis or model. (Classes - e.g DT, linear model, NN).
- **Performance Measure**: Test hypothesis on new set (test set). Metrics:
  - (Regression) Mean Squared Error (MSE), Mean Absolute Error (MAE).
  - (Classification) Accuracy: Accuracy $= \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}(\hat{y}^{(i)} = y^{(i)})$.
  - (Classification) Confusion Matrix: Precision ($\frac{TP}{TP+FP}$ - opt if FP costly), Recall ($\frac{TP}{TP+FN}$ - opt if FN costly), F1 Score: $\frac{2}{\frac{1}{P}+\frac{1}{R}}$
- **Generalization**: Model performance on unseen data. Depends on train dataset quality/quantity (relevance, noise, balance), model complexity.
  - **Bias**: Error stems from model too simple. Model w. high bias fails to capture patterns in data, leading to underfitting. Error does not reduce with more data.
  - **Variance**: Error stems from model too sensitive to small fluctuations in training data. Captures noise, lead to overfitting. Error reduces more data.
  - Simple models tend to have high bias, low variance while complex models have low bias, but high variance. Otherwise, data with significant outliers , weak classifiers, etc can affect bias, variance as well.
- **Hyperparameter Tuning**: Adjust params (e.g. γ) to optimize performance. Predefined before training. Hyperparam search: grid (exhaustive), random etc.

6. Decision Trees

Function input: vector of attribute values. Outputs single value (decision). Perform sequence of tests from root node to leaf.

- **Expressiveness**: Can express any (propositional) fn of input attributes.
- **Size of Hypothesis Class**: Distinct decision trees, n bool. attributes: 2^{2^n}
- **Decision Tree Learning**: Greedy, top-down, recursive algorithm. Use information gain, choose best attribute to divide training set. For each value of chosen attr. use DTL again on corresponding subset of examples.
- **Entropy**: $I(P(v_1), P(v_2), ..., P(v_k)) = -\sum_{i=1}^{k} P(v_i) \log_2 P(v_i)$
- **Information Gain**: Initial Entropy $-$ avg. entropy after divide 'remainder'. Entropy before dividing: $I(\frac{p}{p+n}, \frac{n}{p+n})$, $p+n$ examples in subset E_i.

Information Gain

Given *attribute* A with v distinct values, the training set is split into subsets $E_1, ..., E_v$, each with p_i positive and n_i negative examples. Choose attribute with max information gain.

- remainder$(A) = \sum_{i=1}^{v} \frac{p_i+n_i}{p+n} I\left(\frac{p_i}{p_i+n_i}, \frac{n_i}{p_i+n_i}\right)$
- Info Gain: $IG(A) = I(p_{\text{pos}}, p_{\text{neg}}) - \text{remainder}(A)$

Decision Tree Overfitting / Generalization:

- **Prune Decision Tree**: Smaller tree may have higher accuracy due to noise ignored.
  - **min-sample leaf** (min threshold of sample count in leaf node)
  - **max depth** (longest path from root to leaf) pruning.
- **Data Preprocessing**: Missing values (assign most common value, drop row/attribute etc.) Continuous values - partition to discrete set as attribute.

7. Linear Regression → Regression (Prediction) Tasks

Linear Model: Input vector x of dimension d, hypo class is set of linear fns:

$$h_w(x) = w_0 x_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d \ = w^T x.$$

where w_0(← bias term), $w_1, \ldots, w_d$ are weights, $x_0 = 1$, dummy variable.

$$X = \underset{\text{Design Matrix}}{\begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \ldots & x_d^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \ldots & x_d^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(N)} & x_2^{(N)} & \ldots & x_d^{(N)} \end{bmatrix}} \quad Y = \underset{\text{Target Vector}}{\begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(N)} \end{bmatrix}} \quad w = \underset{\text{Weight Vector}}{\begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}} \in \mathbb{R}^{d+1}$$

- $X \in \mathbb{R}^{N \times (d+1)} | w \in \mathbb{R}^{d+1} | Y, \hat{Y} \in \mathbb{R}^N$ (target, predicted values).
- **Feature Transformations**: modify features for better modeling. Feature Engineering (e.g. poly $z = x^k$/ log $z = \log(x)$/ exp). Feature scaling (e.g. min-max scaling $z_i = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$, standardization $z_i = \frac{x_i - \mu_i}{\sigma_i}$).
- **Measure Fit**: Loss fn of weights w, find w to min. loss. (N : # train pts)
- **Loss Fn**: $J_{MSE}(w) = \frac{1}{2N}\sum_{i=1}^{N}(h_w(x^{(i)}) - y^{(i)})^2$.
- **Derivative**: $\frac{\partial J_{MSE}(w)}{\partial w_j} = \frac{1}{N}\sum_{i=1}^{N}\left(h_w(x^{(i)}) - y^{(i)}\right)x_j^{(i)}$

Learning via Normal Equation: Solve for w that minimizes J_{MSE}: $w = (X^T X)^{-1} X^T Y$. However, problem that computational cost: $O(d^3)$ for matrix inversion. Non-linear models cannot be solved using normal equations.

Learning via Gradient Descent:

- **Algorithm**: Initialize w randomly. Update weights iteratively.
  Repeat until convergence: $w_j \leftarrow w_j - \gamma \frac{\partial J(w_0, w_1, \ldots)}{\partial w_j}$.
- **Learning rate** $\gamma > 0$: Controls step size. (Hyperparameter). can ↓ w time.
- **Variants**: Batch, use all data points. Mini-batch, use data subsets. Stochastic, uses one data point per iteration.
- **Convex**: MSE loss fn. convex, single global min, for linear/poly regression.

8. Logistic Regression → Classification Tasks

Logistic Regression Model: Hypo class: Set of "squashed" linear fns.

- **Sigmoid function**: $\sigma(x) = \frac{1}{1+e^{-x}}$, $\quad \frac{\partial \sigma}{\partial x} : \sigma'(x) = \sigma(x)(1 - \sigma(x))$
- Logistic model: $h_w(x) = \sigma(w_0 x_0 + w_1 x_1 + \cdots + w_d x_d) = \sigma(w^T x)$ where $w_0, w_1, \ldots, w_d$ are weights, $x_0 = 1$ is a dummy variable.

Classification with Logistic Regression: Regression outputs value in $[0, 1]$, treat as probability of class 1. (E.g. $h_w(x) = 0.8$ predicts 80% chance of failure). Use **Decision threshold** (e.g. 0.5) for classification.
Desn. Boundary: surface (e.g. 2D line) separating classes in feature space.
Non-Linearly Separable Data: If single linear decision boundary cannot separate classes, use non-linear models or apply non-linear feature transformations.
Evaluation Metric: Use Accuracy. Don't use BCE, MSE, MAE, (loss functions).

Logistic Regression Loss Function

Use **Binary Cross Entropy** for Loss. MSE not ok for logistic due to non-convexity.

- $BCE(y, \hat{y}) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$.
- **BCE** loss is convex for logistic regression. Small loss if $y \approx \hat{y}$, large loss if y and $\hat{y}$ differ - good loss function.
- **Derivative**: $\frac{\partial J_{BCE}(w)}{\partial w_j} = \frac{1}{N}\sum_{i=1}^{N}(h_w(x^{(i)}) - y^{(i)})x_j^{(i)}$. (same-linear)
- **Weight update** rule: $w_j \leftarrow w_j - \gamma \frac{\partial J_{BCE}(w)}{\partial w_j}$

Multi-Class Classification (extending Binary classification)

Extend binary classification to multiple class by breaking down into sets.
One-vs-one: Binary classifier for every pair of classes $total = \frac{C(C-1)}{2}$ classifiers. Class with most votes wins.
One-vs-rest: Binary classifier for each class, treat all other classes as combined class. C **classifiers.** Class with highest probability output wins.

Regularization (Technique to reduce overfitting, penalize large weights in model.)

- **Overfitting** happens when model too complex, captures training data noise over underlying pattern leading to poor generalization on unseen data.
  - Often associated w. models w. large weights. E.g. Complex polynomial model may fit train data perfectly but fail to generalize to new data. Large oscillations in model may occur as weights w increase, even while fitting same set of points.

Regularization: Add penalty term to loss function, discourage large weights.

- Perform feature scaling before regularization. (Same effect on all variables).
- Given a loss function $J(w)$, add penalty function $P(w)$, hyperparam: $\lambda \geq 0$.

$$\min_w J_{\text{reg}}(w) = J(w) + \lambda P(w)$$

Penalty Functions

- **L2 (Square, Ridge) penalty/regularizer**: Penalize square of weights. *Heavily penalize larger parameters*, but *diminishing return* for reducing already small parameters. Shrink params, but not to 0.
- **L1 (Absolute, Lasso) Regularizer**: Penalize absolute weights. Since penalty grows at same rate, if parameter loss reduction not much, optimal sets weight to 0. Effectively does feature selection strategy, by driving some weights to 0.

$$P_{\text{square}}(w) = \frac{1}{2}\sum_{j=0}^{d} w_j^2, \quad P_{\text{abs}}(w) = \sum_{j=0}^{d} |w_j|,$$

- **Others**: E.g. max-norm penalty, constraint max weight to prevent extreme values.

Regularization: Effect on Learning Algorithms

- **Gradient Descent**: Gradient of regularized loss function has both gradient of original loss, and gradient of penalty term: $\frac{\partial J_{\text{reg}}(w)}{\partial w_j} = \frac{\partial J(w)}{\partial w_j} + \lambda \frac{\partial P(w)}{\partial w_j}$.
- **Normal Equation**: For linear regression with L2 regularization, the closed-form solution becomes: $w = (X^T X + \lambda I)^{-1} X^T Y$, where I is a identity matrix. This ensures invertibility of $X^T X + \lambda I$, even for multicollinearity or insufficient data.

Regularization Strength λ Tradeoffs: Hyperparameter (penalty strength) λ controls trade-off between fitting training data and keeping weights small:

- Small λ: Emphasizes fitting the data, potentially leading to overfitting.
- Large λ: Emphasizes smaller weights, potentially leading to underfitting.

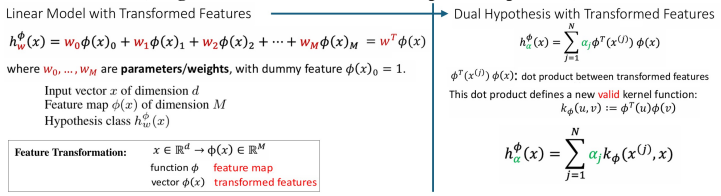
Kernels: Kernel Method

In linear models, weights are linear combinations of inputs (consider how weights calculated through normal equation $X^T Y$), relying only on dot products.
The **kernel method** replaces these dot products with a kernel function $K(x, x')$, enabling powerful nonlinear models without explicit feature mapping.

- **Dot Product and Similarity**: Dot product $[u \cdot v = u^T v]$ measures similarity between 2 vectors. E.g. orthogonal vectors dot product of zero: $\begin{bmatrix}1\\1\end{bmatrix} \cdot \begin{bmatrix}1\\-1\end{bmatrix} = 0$
- **Dual Formulation of Linear Regression:** Linear model $h_w(x) = w^T x$, dual form is where α_j are new parameters (weights), and $x^{(j)}$ are training data points. (Predictions as weighted sum of dot products of training data $x^{(j)}$ and input x).
- **Dual Formulation with Kernel (Kernel Trick):** We simply replace the dot product $(x^{(j)})^T x$ with the kernel function $\mathbf{k(x^{(j)}, x)}$.

$$h_\alpha(x) = \sum_{j=1}^{N} \alpha_j (x^{(j)})^T x \quad \Longrightarrow \quad h_\alpha(x) = \sum_{j=1}^{N} \alpha_j k(x^{(j)}, x)$$

Kernel Trick (Usage of Kernel Method to avoid explicit computation)



Kernel function $k(u, v)$ generalizes dot product, measures similarity efficiently in potentially higher-dimension space, w/o explicit computing transformation.

- **Valid kernel functions**: satisfies conditions like symmetry, positive definiteness.
- **Kernel Machine**: A model that uses the kernel trick.

Kernels and Transformed Features

There is a deep connection between kernels and feature transformations (be it single variable, multi-variable transformations to create new features). Specifically:

- Any valid kernel $k(u, v)$ corresponds to a feature map (transform) ϕ s.t.:

$$k(u, v) = \phi(u)^T \phi(v).$$

- Conversely, any feature transformation ϕ induces a valid kernel.

Examples of Kernels

- **Polynomial Kernel**: $k(u, v) = (u^T v)^s$ where s is degree of polynomial.
- **Gaussian (RBF) Kernel**:

$$k(u, v) = \exp\left(-\frac{\|u - v\|^2}{2\sigma^2}\right)$$

  - where σ^2 controls width of Gaussian ($\downarrow \sigma$ = narrower, $\uparrow \sigma$ = wider).
  - The RBF kernel corresponds to an infinite-dimensional feature map. Computing $\phi(x)$ explicitly may be infeasible, but evaluating $k(u, v)$ efficient.
- **String Kernel**: Measures similarity between strings.
- **Chi-Squared Kernel**: Useful for histograms.
- **Tanh Kernel**: Similar to neural network activation functions.

9. Support Vector Machines (SVM)

SVM is a powerful classifier that can use kernel tricks (allowing it to handle non-linear decision boundaries naturally) - a kernel machine.

- **Decision boundary**: defined by hyperplane separating points of diff. classes.
- **Margin**: distance between hyperplane and closest data points from each class. The width of the margin is proportional to 2 × margin.
- **Maximizes Margins**: SVM model optimized by maximizing margin, subject to constraints that data points are on the correct side of the decision boundary.
- **Support Vectors**: Data points that lie exactly on boundary of margin. These points define position and orientation of hyperplane.

SVM: Data, Model and Sparsity

- Given N data points, described by features $x^{(i)}$, target $y^{(i)} \in \{-1, 1\}$.
- The SVM classifier in dual formulation depends on parameters α_i:

$$\text{SVM}_\alpha(x) = \text{sign}\left(\sum_{i=1}^{N} \alpha_i k(x^{(i)}, x)\right),$$

- **Sparsity**: Many $\alpha_1, \alpha_2, \ldots, \alpha_N$ will be zero. Only support vectors contribute to classifier. Non-support vectors discarded without affecting model.

$$\text{SVM}_\alpha(x) = \text{sign}\left(\sum_{i \in \text{Support Vectors}} \alpha_i k(x^{(i)}, x)\right).$$

Hyperplanes (FYI)

Zero-offset hyperplane: define by normal w, contains all vectors u: $w^T u = 0$.

- **Side**: Sign of $w^T u$ determines which side of hyperplane a point u lies.
- **Distance to Hyperplane:** Distance of a point u to hyperplane given by: Distance $= \frac{|w^T u|}{\|w\|}$ where $\|w\|$ is the length of the normal vector w.

10. Unsupervised Learning (Clustering, Dimensionality Reduction)

- **Unsupervised Learning**: Deals with *datasets w/o ground truth output*. Obtaining ground truth may be expensive/ infeasible. E.g. images w/o predefined categories.
- **Applications**: Clustering, dimensionality reduction, image segmentation, GPT (unsupervised learning component), anomaly detection etc.
- Given set of N data points, $\{x_1, \ldots, x_N\}$, goal is to learn patterns in data.

Clustering

Unsupervised ML technique to group similar data points into clusters. Common applications include data segmentation, anomaly detection.

- **Number of clusters**: User defined, not defined by data.
- **Approaches**: Distance/Partitioning-based clustering (**K-means**, K-Medoids), Hierarchical (agglomerative / divisive), Density-based Clustering (DBSCAN), Distribution-based, Spectral, Grid-based, GMM, Fuzzy etc.

K-Means Clustering

Partitions N data points into K distinct clusters. Each cluster represented by **centroid**, the mean of all points in the cluster.

K-Means Algorithm

1. **Initialize**: Randomly initialize K centroids $\mu_1, \ldots, \mu_K$.
   (a) **Assign each Point to Cluster**: $\forall x^{(i)}$, assign to nearest centroid: $c^{(i)} = \arg\min_k \|x^{(i)} - \mu_k\|^2$ ($c^{(i)}$ is cluster assignment for $x^{(i)}$)
   (b) **Update Centroids**: *Re*-compute centroids as mean of all points assigned to cluster: $\mu_k = \frac{1}{N_k}\sum_{i \in C_k} x^{(i)}$
   (C_k is set of points in cluster k, N_k is number of points in C_k)
2. **Convergence**: The algorithm always terminates, when assignments no longer change or centroids stabilize. Each K-means step never increases distortion.

- **Measuring the Quality of Clusters**: (use distortion function)
  Computes average squared distance of each point to its assigned centroid:

$$J(c^{(1)}, c^{(2)}, \ldots, c^{(N)}, \mu_1, \mu_2, \ldots, \mu_K) = \frac{1}{N}\sum_{i=1}^{N} \|x^{(i)} - \mu_{c^{(i)}}\|^2.$$

- **K-Means Optimizations**:
  - **Mean normalization & Feature Scaling**: prevent cost function J from being dominated by certain select features.
  - **Multiple random restarts**: K-means prone to local optima. Random restarts' random initialization with further optimizations, such as probalistic method to choose next centroid, or choose next centroid as far as possible.
- **Picking Clusters Count (K)**: **Elbow Method**, plot distortion J against K, 'elbow' point is where adding more clusters yields diminishing returns.
- **Variants of K-Means**: *K-Medoids*: Actual data points as cluster reps (medoids), or *Snap to Nearest Data Point*: Centroids "snapped" to nearest data points., etc.

Kernel K-means Clustering

Use the kernel trick in distance computation in K-means clustering. E.g. use Gaussian radial basis function (RBF) kernel, map data into higher-dimensional space, non-linearly separable clusters potentially become linearly separable. (kernel trick applied only to dot products between data points, not point and cluster mean).

Dimensionality Reduction (Of dataset)

Reduce number of features in dataset while retaining maximal relevant information.

- **Curse of Dimensionality**: High-D data (e.g. HD images - $1280 \times 720 = 921,600$ features). Exponential increase in sample complexity with number of features.
- $N = O(2^d)$, where d is # features, N is # samples required for effective learning.
- **Dimension Reduction**: Identify, remove redundant feature by changing basis.

Singular Value Decomposition (SVD) (Matrix factorization technique)

Let $d > N$: for any $d \times N$ real-valued matrix A, there exists a factorization:

$$A = U\Sigma V^T,$$

- $U \in \mathbb{R}^{d \times d}$: Matrix of left singular vectors (new basis) w. orthonormal cols.
- $\Sigma \in \mathbb{R}^{d \times N}$: Diagonal matrix of singular values (importance of each basis vector). Singular values are ordered such that: $\sigma_1 \geq ... \geq \sigma_r \geq 0$.
- $V \in \mathbb{R}^{N \times N}$: Matrix of right singular vectors (coefficients for linear combinations) with orthonormal columns and rows.
- **Interpretation of SVD**: SVD provides new basis for data:
  - Columns of U form an orthonormal basis for the row space of A.
  - Columns of V form an orthonormal basis for the column space of A.
  - Singular values σ_j: importance of basis vector in capturing data variance.
- **Dimensionality Reduction via SVD**: As singular values σ_j indicate importance of corresponding basis vectors, to reduce dimensionality, retain only top r singular values $(\sigma_1, \ldots, \sigma_r)$. Truncate Σ to $\tilde{\Sigma}$, keep first r rows, columns. Use reduced basis matrix $\tilde{U}$ (first r columns of U) to project data into a lower-dimensional space.
- **Compression**: Original data matrix X^T approximated using truncated SVD:

$$\text{original: } X^T = U\Sigma V^T, \quad \text{so approx: } \tilde{X}^T = \tilde{U}\tilde{\Sigma}\tilde{V}^T$$

$$\tilde{X}^T = \tilde{U}Z = \tilde{U}\tilde{U}^T X^T \approx X^T. \quad (\tilde{U}\tilde{U}^T \approx I)$$

where $\tilde{U} \in \mathbb{R}^{d \times r}$, $\tilde{\Sigma} \in \mathbb{R}^{r \times r}$, and $\tilde{V} \in \mathbb{R}^{N \times r}$, $(X^T / \tilde{X}^T \in \mathbb{R}^{d \times N})$.
where $\tilde{U}$: reduced left singular vectors, Z: compressed rep of data.

- **Example: Lossy Data Compression**: Hence, given data matrix X^T, can compress data by projecting it onto new reduced (lower-dimensional) basis $\tilde{U}$:

$$Z = \tilde{U}^T X^T, \quad Z \in \mathbb{R}^{r \times N}$$

Intuition: Project data points $x^{(i)}$. Instead of explicitly working w. $\tilde{\Sigma}, \tilde{V}^T$, project data onto reduced basis $\tilde{U}$. Z contains all projection coefficients.

- **Assumption of Mean-Centered Data**: Data matrix X^T assumed to be mean-centered (each feature has zero mean). Ensure singular values σ_j^2 directly represent variances along principal directions (axes) defined by singular vectors $u^{(j)}$, simplify interpretation of SVD results.
- **Retained Variance**: Reconstruction error depends on retained singular values. To retain at least 99% of variance, choose smallest r such that:

$$\text{Retained Variance} = \frac{\sum_{i=1}^{r} \sigma_i^2}{\sum_{i=1}^{N} \sigma_i^2} \geq 0.99.$$

- **Applications of SVD**: Noise reduction. Data visualization (Project data to 2D).
- **Practical implementation**: SVD available using popular numerical computing libraries (*Numpy*: `numpy.linalg.svd`, *Matlab*: `svd`)

Principal Component Analysis (PCA) (Application of SVD)

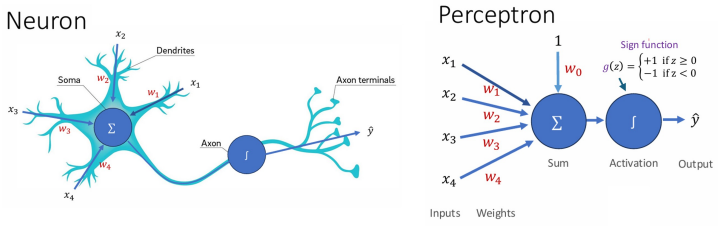
- PCA identifies directions (principal components) that capture maximum variance in data. Principal components correspond to largest singular values in SVD
- By projecting data onto top r principal components, PCA achieves dimensionality reduction while retaining most important information.

11. Neural Networks

Perceptron (Binary Classifier)

Binary classifier: learns to separate data into two classes based on input features. Fundamental NN building block, inspired by biological brain neurons.

- **Biological neuron**: receives inputs (dendrites), produce output (axon).
- **Perceptron**: takes input features, applies weights, takes summation, passes result through an activation function to produce an output.



Perceptron Model

For input vector $\mathbf{x}$ of dimension d, perceptron model defined as:

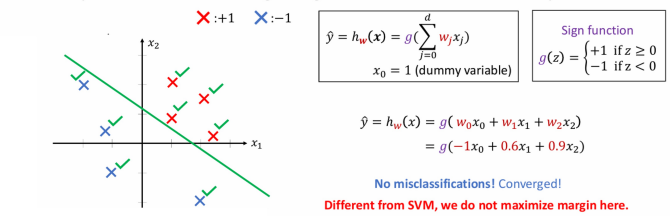
$$h_w(x) = \hat{y} = g(\mathbf{w}^T\mathbf{x}) = g\left(\sum_{j=0}^{d} w_j x_j\right), \quad g(z) = \begin{cases} +1 & \text{if } z \geq 0, \\ -1 & \text{if } z < 0. \end{cases}$$

- Equivalently, $h_w(x) = g(w_0 x_0 + w_1 x_1 + ... + w_d x_d)$
- $\mathbf{w} = [w_0, w_1, \ldots, w_d]$ are the weights (parameters) of the perceptron.
- $x_0 = 1$ is a dummy variable (bias term).
- $g(z)$ is the activation function, typically a step function.
- N **data points**, each has **Features**: $x^{(i)} \in \mathbb{R}^d$ **Target**: $y^{(i)} \in -1, 1$
- Features are described by a vector of real numbers in dimension d (i.e. x)

Perceptron Learning Algorithm (Iteratively update weights to train model)

1. Initialize the weights $\mathbf{w}$ (e.g., to zero or small random values).
2. For each training instance $(\mathbf{x}^{(i)}, y^{(i)})$:
   - Compute predicted output $\hat{y}^{(i)} = g(\mathbf{w}^T\mathbf{x}^{(i)})$
   - If prediction incorrect $(\hat{y}^{(i)} \neq y^{(i)})$, i.e. misclassified, update weights: $[\mathbf{w} \leftarrow \mathbf{w} + \gamma \cdot (y^{(i)} - \hat{y}^{(i)}) \cdot \mathbf{x}^{(i)}]$, where γ is the learning rate.
3. Repeat until convergence or a maximum number of iterations is reached.

Example: Cancer Prediction: Binary classification of tumor.



Perceptron Limitations

- Solves linearly separable problems, cannot handle non-linear feature relationships.
- To address these limitations, multi-layer neural networks are used, which stack multiple perceptrons (neurons) with non-linear activation functions.

Neural Network

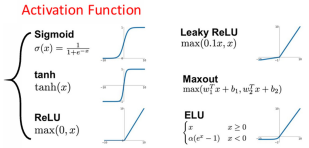
Multiple layers of interconnected neurons, model complex/non-linear patterns.

- **Applications**: Neural networks widely used for image classification, NLP (sentiment analysis), multi-class classification (predict out of several categories).

Neuron (Generalization of a Perceptron)

An artificial neuron generalizes perceptron using diff. activation functions:

$$\hat{y} = h_w(x) = g\left(\sum_{j=0}^{d} w_j x_j\right)$$



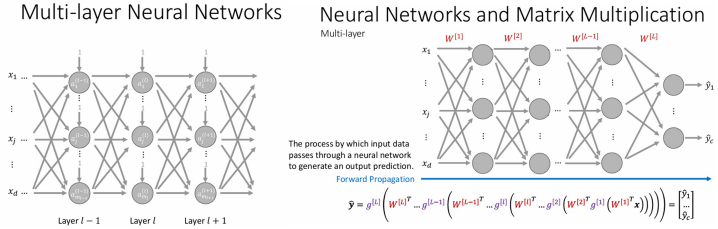
- A neuron w. linear activation function equivalent to linear regression model.
- A neuron w. sigmoid activation function eqv to logistic regression model.
  - Identity/Linear activation fn: $g(z) = z \implies h_w(x) = \sum_{j=0}^{d} w_j x_j$
  - Sigmoid function: $g(z) = \sigma(z) \implies h_w(x) = \sigma(\sum_{j=0}^{d} w_j x_j)$

Feature Engineering (To model Non-Linearly Separable Problems)

- **Manual Feature Engineering**: Manual transform existing features to capture non-linear relationships, e.g. polynomial features: $x_1^2, x_2^2, x_1 x_2$, exponential or logarithmic transformations: $e^{x_1}, \log(x_2)$, combine features multiplicatively: $x_1 x_2$.
- **Using Neurons for Feature Transformation**: Use neuron layers to auto learn new features. Hidden layers apply non-linear activation functions (e.g., sigmoid, ReLU). Weights of neurons effectively create new features making it linearly separable.
- **Feature Engineering Usage**: manual feature engineering used in Linear, Logisitic Regression to capture complex patterns. Multi-layer neural network learns on own.
- E.g. To model XOR or XNOR gate (not linearly separable), require intermediate hidden layer for preliminary transformation to achieve logic of XOR/XNOR gate.

Multi-layer Neural Networks

Three components: *input layer* (raw features), *hidden layers* (transform features w learned weights, activation functions), and *output layer* (final predictions).



The forward propagation process computes the output as:

$$\hat{\mathbf{y}} = g^{[L]}\left(W^{[L]} \cdot g^{[L-1]}\left(W^{[L-1]} \cdot \ldots g^{[1]}\left(W^{[1]} \cdot \mathbf{x}\right)\ldots\right)\right),$$

where $W^{[l]}$ are the weights for layer l, and $g^{[l]}$ is the activation function.

Multi-class Classification: Softmax Function As Activation Function

For multi-class classification, the softmax function is used in the output layer to convert raw scores into probabilities:

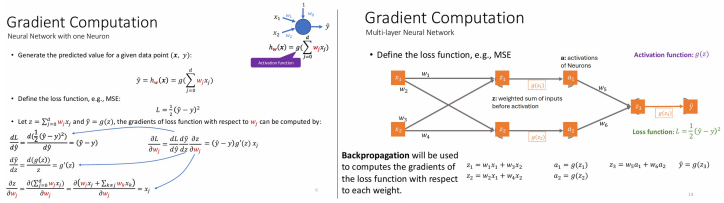
$$\textbf{SoftMax:} \quad g(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{c} e^{z_j}},$$

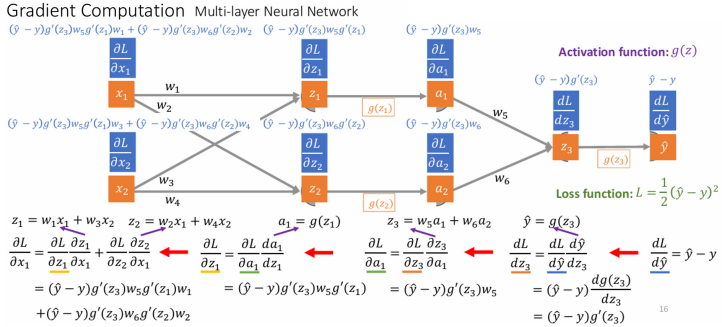
where c is the number of classes, and $g(z_1) + g(z_2) + \cdots + g(z_c) = 1$.

- **Softmax function**: used to convert raw scores z_i (summation before activation) into probabilities, and ensures that output values sum to 1. (Normalizes output to probability distribution over predicted output classes.)
- **Usage**: Set number of neurons in last layer as c, predict probability of each class using softmax and pick highest probabilty.

Neural Networks Training (Adjusting weights to minimize loss)

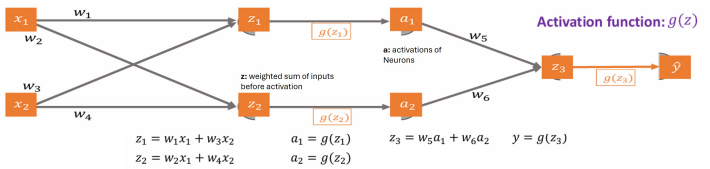
Gradient Descent





Forward Propagation (Input data passed through to compute output)

- $\hat{y} = g(W^{[L]} \cdot g(W^{[L-1]} \cdot \ldots g(W^{[1]} \cdot x) \ldots))$
- $W^{[l]}$: Weight matrix for layer l, x: Input features. $g(\cdot)$: Activation function.
- For example, in a simple multi-layer neural network:



Loss Function: quantifies error between predicted output $\hat{y}$ and the true target y. Common loss functions include Mean Squared Error (MSE) ($L = \frac{1}{2}(\hat{y} - y)^2$), Cross-Entropy Loss (for classification tasks).

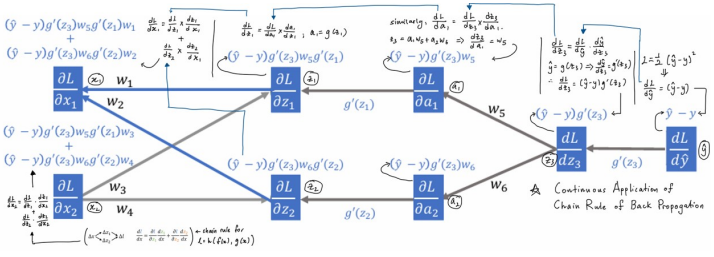
Backpropagation (Gradient Computation backwards)

Compute gradients of loss function w.r.t weight w. chain rule (MSE Loss):

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_j}. \quad \leftrightarrow \frac{\partial L}{\partial \hat{y}} = \hat{y} - y. \quad \frac{\partial \hat{y}}{\partial z} = g'(z). \quad \frac{\partial z}{\partial w_j} = x_j.$$

- $L = \frac{1}{2}(\hat{y} - y)^2$, so $\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$, and $\hat{y} = g(z)$, so $\frac{\partial \hat{y}}{\partial z} = g'(z)$.
- e.g. $z_1 = w_1 x_1 + w_3 x_2$, so: $\frac{\partial z_1}{\partial w_i} = x_j$

Key steps in backpropagation: Compute derivative of loss wrt. output y. Compute derivative of activation function. Propagate error backward through layers, compute respective derivatives. Then compute gradients $\forall$ weights.



Gradient Descent: Weight Update Formula (Using computed gradients)

$$w_j \leftarrow w_j - \gamma \frac{\partial L}{\partial w_j}, \quad \gamma: \text{ learning rate.}$$

- **E.g.**: $\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_3} = [(\hat{y} - y) \cdot g'(z_3) \cdot w_5 \cdot g'(z_1)] \cdot [x_2]$
- **Iterative Optimization**: Forward propagation, loss computation, backpropagation, and weight update steps repeated iteratively until convergence / stopping criterion.

Single-Layer Neural Network (One Neuron)

Single neuron as simplest form of neural network. Performs operations:

1. **Weighted Sum**: $z = \sum_{j=0}^{d} w_j x_j = w^T x$, ($w_0$: bias term).
2. **Activation Function**: $\hat{y} = g(z)$, e.g. sigmoid fn: $g(z) = \frac{1}{1+e^{-z}}$

- For **Binary classification**, activation function is typically sigmoid function. Then, the loss function for a single neuron is often the binary cross-entropy (BCE) loss: $L = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$.
- **Binary Classification Hyperparameters** α, β usable as weights ($L = \alpha[-y \log(\hat{y})] + \beta[-(1 - y) \log(1 - \hat{y})]$. Can mitigate problem of highly unbalanced dataset, so machine wont bias towards predicting all samples as overrepresented class. (Punish misclassification of underrepresented class more)
- **Optimization**: For single layer NN, could do manual feature engineering. However, could be computationally expensive, with overfitting if too many features. Final decision still ultimately a linear combination of transformed features.

Multi-layer Neural Network

Multiple layers of neurons, enables modeling complex, non-linear relationships.

- **Hidden Layers**: Hidden layers transform the input features into higher-level representations:

$$a^{[l]} = g(W^{[l]} a^{[l-1]} + b^{[l]}),$$

where $a^{[l]}$ is the activation of layer l.

- **Activation Functions**: Includes ReLU (Rectified Linear Unit): $g(z) = \max(0, z)$, Sigmoid: $g(z) = \frac{1}{1+e^{-z}}$, Softmax (see multi-class classification).
- **Output Layer**: produces final prediction. For regression tasks, it uses a linear activation function. For classification tasks, it uses softmax or sigmoid.
- **Optimizations of Deep NN**: For a multi-layer NN:
  - **Increase Depth of Network**: Add more hidden layers.
  - **Increase Width of Network**: Add more neurons per layer.
  - Complexity of computed function grows exponential with depth, increase more slowly with width. Increasing depth generally more efficient.
  - Increased capacity of learning complex patterns also increases risk of overfitting.
  - **Others**: All weights not equal (initial layer matters more, more sensitive), lower layer weights less robust to noise but perform better when optimized. (See paper)
- **Potential Issues of Deep NN**:
  - **Sigmoid Function**: to compute derivative of first few layers (e.g. $\frac{\partial \varepsilon}{\partial W^{[1]}}$), it is product of many derivatives, which is each between 0 and 1. Known as **Vanishing Gradient problem**.
  - To mitigate, we can use ReLU function, as long as input is non-negative, derivative is 1, precluding the issue.
  - **ReLU function**: When majority of activations are 0 (underlying pre-activation summation mostly non-positive) - network dies halfway. Gradients passed back are 0, cannot recover, stops learning. Known as **Dying ReLU problem**.
  - To mitigate, use Leaky ReLU, has small positive gradient for negative inputs, gives network chance to recover.

12. Convolutional Neural Networks (CNNs)

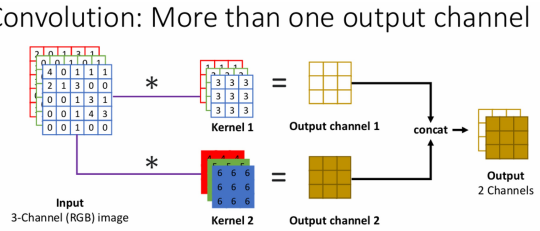
CNNs: specialized neural networks for process grid-like data (e.g. image).

- Leverage convolution operations to extract spatial features efficiently.
- **Convolution**: math operation, combines 2 functions to produce third function.
  - One function represents the input data (e.g., an image).
  - Other function represents a small matrix called a filter or kernel
  - The convolution operation involves sliding the filter over the input and computing the weighted sum at each position. This process extracts spatial features from the input, such as edges, textures, or patterns.

Convolution Layer

Applies filters to extract spatial features: Output = Input $*$ Filter + Bias.

- **PyTorch**: `conv_layer = nn.Conv2d(in_channels, out_channels, kernel_size, stride, padding)`



conv_layer = torch.nn.Conv2d(in_channels=3, out_channels=2, kernel_size=3, stride=1, padding=0)

Example of Convolution Operation: Consider 2D input X, kernel W:

$$X = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}, \quad W = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}. \quad \text{FM} = \begin{bmatrix} -3 & 4 \\ -3 & 4 \end{bmatrix}.$$

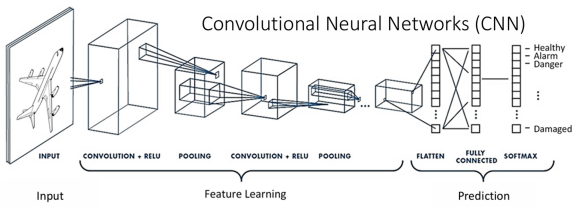
The convolution operation involves sliding the kernel over the input and computing the weighted sum at each position, obtaining Feature Map (FM).

- **Padding**: Adds zeros around the input to control the size of the output.
- **Stride**: Controls the step size of the kernel as it slides over the input.
- For a single image and a single kernel, output height $H_1 = \lfloor \frac{H-K+2P}{S} \rfloor + 1$.

Pooling Layer

Pooling layers **downsample** feature maps to reduce dimensionality.

- **Max-Pooling**: Takes the maximum value in a window.
- **Average-Pooling**: Computes the average value in a window.
- **Sum-Pooling**: Sums values in a window.



Popular CNN Architectures widely used CNN architectures include:

- **VGG**: Deep networks with small 3×3 filters.
- **Inception**: Parallel convolution operations with different filter sizes.
- **DenseNet**: Dense connections between layers.
- **ResNet**: Residual connections to address vanishing gradients.

CNN Applications: *Image Classification*: Identify objects in images (e.g. face emotion). *Image Segmentation*: Detect regions of interest (e.g. cancer cell). **Object Detection**: Locate / classify objects (e.g. self-drive cars).

13. Recurrent Neural Networks (RNNs) (Processes Sequential Data)

- **Sequential data**, e.g. text, audio, has **patterns, dependencies** across time. Elements treated sequentially to capture relationships.
- **Hidden State**: RNNs maintain a **hidden state**, unlike traditional feedforward networks. It captures info of previous elements in sequence, to encode contextual info and model temporal dependencies.
- **One-hot Encoding**: method for converting categorical variables into a binary format (e.g. convert word to vector w. length equal to vocab size).
- **Time step**: single iteration in RNN, one element of sequence processed.
- **Example**: 'we saw this saw.', use one-hot encoding, represent word as vectors. Predict'saw' is verb or noun requires context of previous words.

RNN Structure

An RNN updates hidden state at each time step while it processes sequential data.

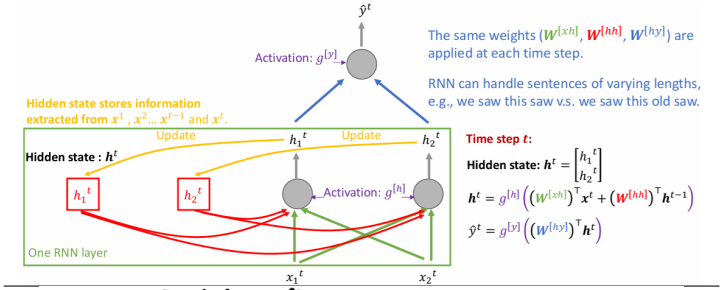
- **Hidden State**: Number of elements in hidden state set according to no. of output we want to generate from each layer (e.g. 2 output $\leftrightarrow$ two elements)
- Hidden state typically initialized to 0 ($h^0 = \mathbf{0}$).
- Same **3 different weight matrices**, ($\mathbf{W}^{[xh]}, \mathbf{W}^{[hh]}, \mathbf{W}^{[hy]}$) applied at each time stamp. (weight for each edge)

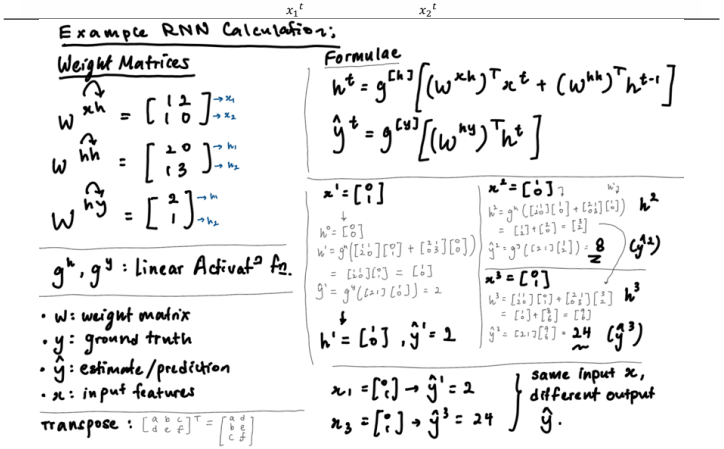
The key equations governing an RNN are:

$$\text{Updating } h: \quad \mathbf{h}^t = g^{[h]}[(\mathbf{W}^{[xh]})^\top \mathbf{x}^t + (\mathbf{W}^{[hh]})^\top \mathbf{h}^{t-1}]$$

$$\text{Predicting Output:} \quad \hat{\mathbf{y}}^t = g^{[y]}[(\mathbf{W}^{[hy]})^\top \mathbf{h}^t]$$

- $\mathbf{x}^t$: Input at time step t, $\mathbf{h}^t$: Hidden state at time step t
- $\hat{\mathbf{y}}^t$: Output prediction at t $\mathbf{g}^{[h]}, \mathbf{g}^{[y]}$: Activation Fn (sigmoid, softmax).

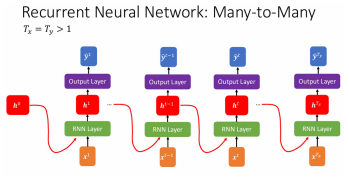


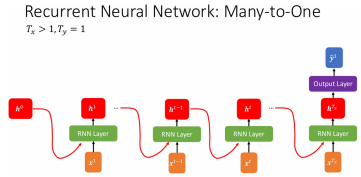


The hidden state $\mathbf{h}^t$ acts as a memory that stores information about past inputs. At each time step, the RNN combines the current input $\mathbf{x}^t$ with the previous hidden state $\mathbf{h}^{t-1}$ to compute the new hidden state.

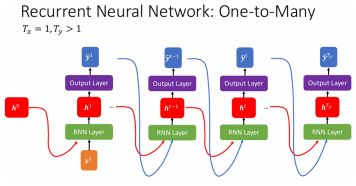
Types of RNN Architectures (Configuration depending on task)

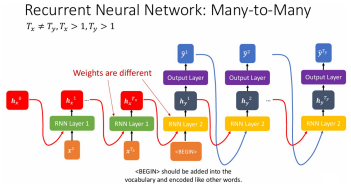
- **Many-to-Many** ($T_x = T_y > 1$): Both input and output sequences have the same length. E.g.: Machine translation (e.g., Eng to French).
- **Many-to-One** ($T_x > 1, T_y = 1$): A sequence of inputs produces a single output. E.g.: Sentiment analysis (review positive or neg).





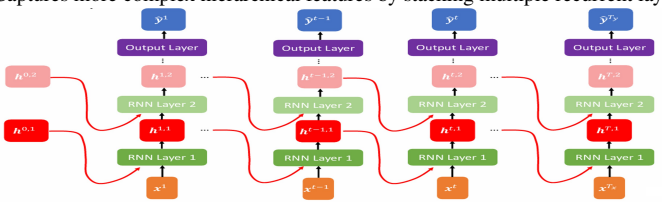
- **One-to-Many** ($T_x = 1, T_y > 1$): A single input generates sequence of outputs. E.g.: Image captioning (e.g. generate image description).
- **Many-to-Many** ($T_x \neq T_y, T_x > 1, T_y > 1$): Input and output sequences have different lengths. Has <begin> token. E.g.: Video captioning.





Deep Recurrent Neural Networks

Captures more complex hierarchical features by stacking multiple recurrent layers.



- **Application of RNNs**: widely used in tasks involving sequential data, such as *Sentiment Analysis*, *Speech Recognition*: Transcribing spoken language into text, *Video Captioning*: Generating descriptions for video frames.
- **Properties of RNN**: *Capture Contextual Information*: RNNs use hidden state to encode information about entire sequence seen so far. **Sequential in Nature**: Predictions at time step t depend on all previous steps, less parallelism-friendly.

14. Attention Neural Networks: Self-Attention (Layer)

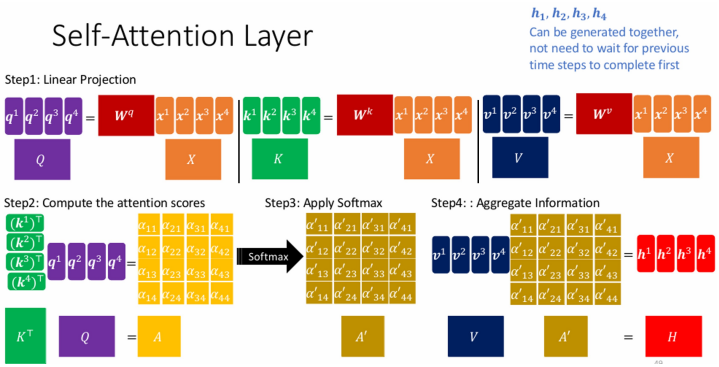
Mechanism that captures contextual information, identify relevant parts of input sequence. Allows model to aggregate useful info from all elements in sequence, to predict current output. $\rightarrow$ can handle sequential data.

- **Query** (Q): Represents current element being processed.
- **Key** (K): Encodes relevance of other elements in sequence to the query.
- **Value** (V): Contains the actual content or information to be aggregated.

Components derived from input sequence using trainable weight matrices:

$$Q = W^q X, \quad K = W^k X, \quad V = W^v X,$$

where X is (**column major input matrix**, each column correspond to single feature vector e.g. $\begin{bmatrix} x_1 & y_1 & z_2 \\ x_2 & y_2 & z_2 \end{bmatrix}$), and W^q, W^k, W^v are the weight matrices for queries, keys, and values, respectively.

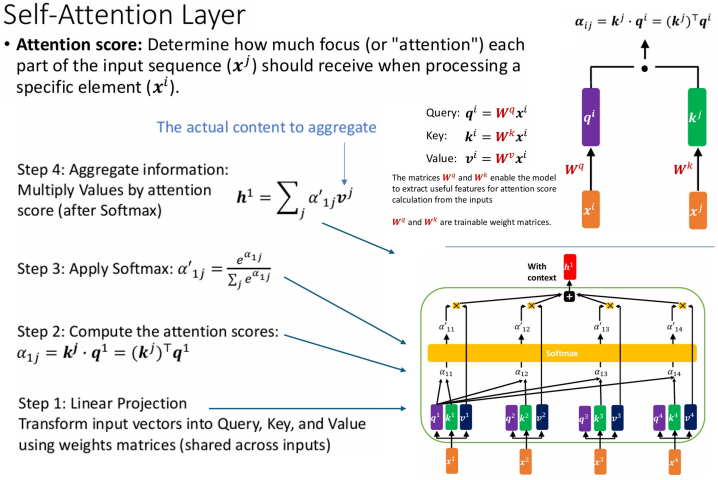


Steps in Self-Attention Mechanism

1. **Linear Projection**: Each input vector x_i is transformed into query, key, and value vectors: $q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i$
2. **Compute Attention Scores**: attention scores measure relevance of each element in sequence to current query: $\alpha_{ij} = k_j^\top q_i$
   k_j: key vector for j-th element, q_i: query vector for i-th element.
3. **Apply Softmax**: attention scores are normalized using softmax function to obtain probabilities: $\alpha'_{ij} = \frac{\exp(\alpha_{ij})}{\sum_j \exp(\alpha_{ij})}$
4. **Aggregate Information**: Final output for i-th element is a weighted sum of value vectors: $h_i = \sum_j \alpha'_{ij} v_j$
   α'_{ij}': normalized attention score, v_j: value vector for j-th element.

Self-Attention Layer

- **Attention score:** Determine how much focus (or "attention") each part of the input sequence ($\boldsymbol{x}^j$) should receive when processing a specific element ($\boldsymbol{x}^i$).



- **Advantages of Self-Attention**: Captures long-range dependencies in input sequence, does not require sequential processing, can parallel compute, context-aware representations for each element in sequence.
- **Applications**: Self-attention core component of transformer architectures, widely used in NLP, (machine translation, text summarization), Computer Vision (image classification etc), Speech Processing (speech recognition, synthesis).
- **Still requires Positional Encoding**: One-hot encoding to represent words, input vectors to self-attention layer, generate outputs. But, **positional encoding** still needed to generate diff outputs for different unique positional instances.

Positional Encoding

Technique to inject positional information into input features, e.g. one-hot vectors.

- **Purpose**: Explicitly encode the position of each token in the sequence.
- **Formula**: The positional encoding vector PE_i for position i is added to the original input feature vector x_i: $[x'_i = x_i + PE_i]$.
  - $x_i \in \mathbb{R}^d$: Original input feature vector for position i,
  - $PE_i \in \mathbb{R}^d$: Positional encoding vector for position i,
  - $x'_i \in \mathbb{R}^d$: Final input feature vector after adding positional encoding.
- **Generating Positional Encoding Using Sine and Cosine Functions**:

  The positional encoding values are computed using sine and cosine functions:

$$PE_{i,2k} = \sin\left(\frac{i}{10000^{\frac{2k}{d}}}\right), \quad PE_{i,2k+1} = \cos\left(\frac{i}{10000^{\frac{2k}{d}}}\right)$$

  - i: Position index (starting from 0), d: Input feature dimension.
  - k: Dimension index (starting from 0, goes up to $\frac{d}{2} - 1$)
  - In this case, Dimension tells whether to use sine or cosine.
- **Example**: Generate Positional Encoding for input feature dimension $d = 4$

$$\text{For } i = 1 : PE_1 = \begin{bmatrix} \sin(1)_{(k=0)} \\ \cos(1)_{(k=0)} \\ \sin\left(\frac{1}{100}\right)_{(k=1)} \\ \cos\left(\frac{1}{100}\right)_{(k=1)} \end{bmatrix} . \text{For } i = 2 : PE_2 = \begin{bmatrix} \sin(2) \\ \cos(2) \\ \sin\left(\frac{1}{50}\right) \\ \cos\left(\frac{1}{50}\right) \end{bmatrix}.$$

- **Positional Encoding Purpose**: Enable self-attention layers to distinguish btw. identical tokens at different positions. (e.g. used with one-hot encoding)
- Positional encoding ensures sequential models capture order of elements in input sequence. Particularly important in architectures like Transformers, which do not inherently process sequences in order.

Deep Learning Issues / Problems

Overfitting

Model fits train data too closely, captures noise, cannot generalize to unseen data.

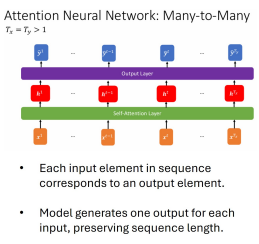
- **Causes of Overfitting**: Model overly complex (e.g., too many params), insufficient training data, training too many epochs w/o proper regularization.
- **Solutions to Overfitting**:
  - **Dropout**: During training, randomly set some neurons' outputs to zero with a given probability. Prevents model from relying too heavily on specific neurons.
  - **Early Stopping**: Monitor the validation loss during training. Stop training when validation loss stops improving, even if training loss continues to decrease. Prevents the model from over-optimizing on the training data.
  - **Regularization**: Add penalty terms (e.g., L1, L2 regularization) to loss function.
  - **Data Augmentation**: Increase size, diversity of training dataset by applying transformations (e.g., rotations, flips, noise).

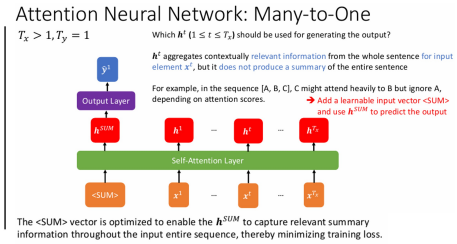
Vanishing and Exploding Gradients (Specific to Backpropagation)

1. **Vanishing Gradient**: Small gradients multiplied repeatedly in backpropagation through layers, tend to 0. Prevents earlier layers from learning effectively.
   - **Mitigation Strategies for Vanishing Gradients**: Use activation functions that avoid saturation (e.g., ReLU instead of sigmoid or tanh), Normalize inputs and use techniques like Batch Normalization to stabilize gradients.
2. **Exploding Gradient**: Large gradients get multiplied repeatedly, causing numerical instability or overflow. Destabilizes training process.
   - **Mitigation Strategies for Exploding Gradients**: Apply gradient clipping: Clip gradients to a specified range (e.g., $[-\text{clip\_value}, \text{clip\_value}]$). Use weight initialization techniques to prevent large initial gradients.

Attention Neural Networks: Types

- **Many-to-Many Attention**: where input and output sequences same length.
- **Many-to-One Attention**: Input sequence processed to produce a single output.
  - **Added Input**: <SUM>, which is learnable input token vector that triggers summarization. It is added to capture summary representation.
  - **Added hidden state**: h_{SUM} is the hidden state that captures summarized sequence, $h_{\text{SUM}} = \sum_j \alpha'_j v_j$, where α'_j are normalized attention weights.
  - Why need new hidden state? Each hidden state h_t only aggregates contextually relevant info, but not summarize entire sequence. h^t is position-specific focused, summary h^{SUM} captures global context.
  - (tasks e.g. sentiment analysis, classification).

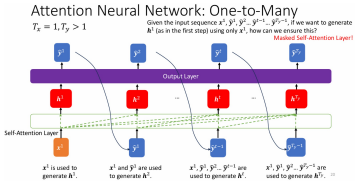


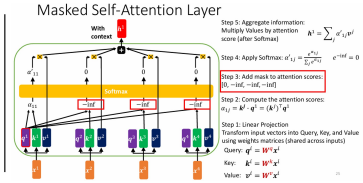


- **One-to-Many Attention**: A single input (x_1) generates sequence of multiple outputs, where $T_x = 1$ and $T_y > 1$ (e.g. text generation).
  - **Algorithm**: Generate 1st output $\hat{y}_1$ via hidden state h_1. Subsequent $\hat{y}_2, \hat{y}_3, \ldots$ generated sequentially, incorporate previous predictions $\hat{y}_1, \hat{y}_2, \ldots$ as input.
  - **Masked Self-Attention**: To ensure model only uses previously generated tokens during prediction, masked self-attention is applied.

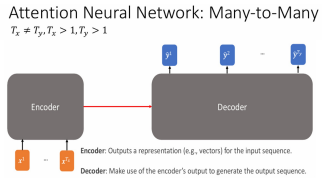
$$\text{Masked Attention Scores} = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}, \text{for softmax function}$$

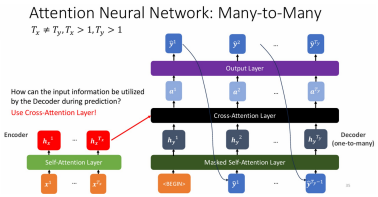
    Diagonal, lower triangular entries set to 0 (allow attention to current, past tokens). Upper triangular set to $-\infty$ (blocking attention to future tokens).
  - **ADD mask** after computing attention scores, prevent attending to future tokens (cheating). Maintains causal prediction, as during training, in techniques like teacher forcing, ground truth tokens ($y_1, y_2, \ldots$) provided.
  - Without mask, model could "cheat" by attending to future tokens, leading to overfitting and poor generalization during inference.



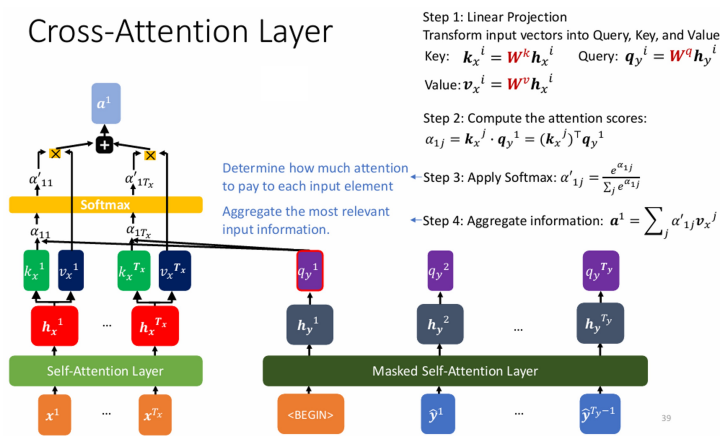


Many-to-Many Attention (input/output sequence different lengths)



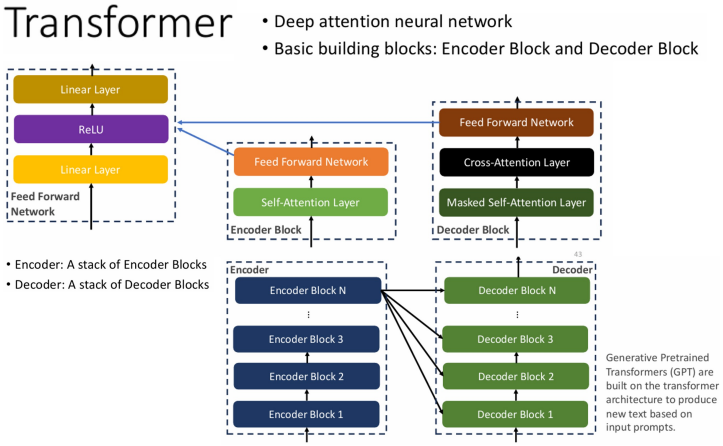


- **Encoder-Decoder Architecture**: Encoder process input sequence, produce contextual representation. Decoder uses repn. to generate output sequence.
- **Cross-Attention Layer**: enables decoder to focus on relevant parts of encoder's output while generating tokens in out sequence. Computes attention scores using:
  - **Query**: (Q): Derived from decoder's hidden states.
  - **Key** (K) **and Value** (V): Derived from the encoder's output.



Transformers (Deep attention-based Neural Networks).

- **Key Characteristics**: Transformer relies entirely on attention mechanisms, eliminating recurrence and convolution. Composed of two main components: the **Encoder** and the **Decoder**, each consisting of multiple stacked blocks.
- **Basic Building Blocks**:
  - **Self-Attention Layer**: Capture r/s btw. all elements in input sequence.
  - **Feed Forward Network (FFN)**: A two-layer neural network with a ReLU activation applied independently to each position.
  - **Masked Self-Attention Layer**: Ensures causality in autoregressive tasks.
  - **Cross-Attention Layer**: Enables decoder to focus on parts of encoder's output.
  - **Linear Layers**: Used for projection and transformation of inputs.



- **Applications of Transformers**:
  - **Generative Pretrained Transformers (GPT)**: Built on the Transformer architecture to generate new text based on input prompts.
  - **Vision Transformers (ViT)**: Adapt Transformers for image classification by treating image patches as sequence elements. (+ Positional Encoding!)
  - **Others**: machine translation, speech recognition, time-series forecasting etc.

15. AI Ethics

- **Ethics**: Principles guiding human behavior, distinguishing right, wrong.
- **AI Ethics**: Principles that guide developers, manufacturers, authorities, and operators in mitigating the ethical issues arising from AI.
- **Key Ethical Issues**:
  - **Biased Decision-Making**: AI can generate biased outputs, make unfair judgments due to biases in training data or algorithms. (E.g. Bias in AI recruiting tools, generative AI)
  - **Manipulation of Behavior**: AI can be used to generate fake content (e.g., deepfakes) to influence human behavior and decision-making.
  - **Automation and Employment**: AI can replace humans in certain jobs, leading to unemployment and economic inequality.
  - **Autonomous Systems**: Autonomous systems can make decisions independently, raisie ethical questions: How should autonomous systems behave, Who is accountable for harmful actions taken.
  - **Singularity**: Singularity in AI refers to a hypothetical future point when artificial intelligence not only reaches but exceeds human intelligence. Will the singularity ever occur? What would happen if it did occur? We still don't know. Poses existential risks and challenges, including the potential loss of human control over advanced AI systems.

16. Appendix

Introduction to PyTorch

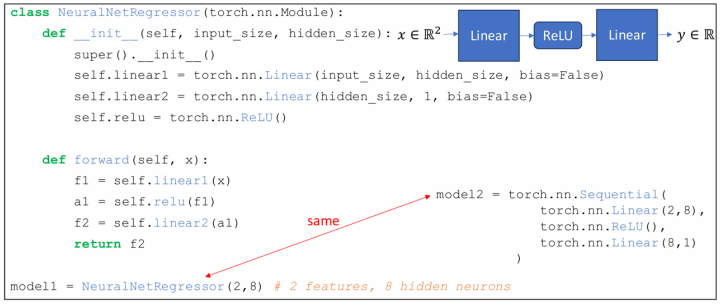
PyTorch: popular deep learning library, simplifies implementation of neural networks. Flexible, efficient framework for building, training models.

Modules and Functions API

PyTorch organizes neural network components into modular containers and layers.

- **Containers**:
  - `torch.nn.Module`: Base class for all neural network modules.
  - `torch.nn.Sequential`: A container for stacking layers sequentially.
- **Linear Layers**:
  - `torch.nn.Linear`: A fully connected layer without activation.
- **Non-linear Activation Functions**:
  - `torch.nn.ReLU`: Rectified Linear Unit.
  - `torch.nn.Sigmoid`: Sigmoid activation function.
  - `torch.nn.Softmax`: Softmax activation function.
- **Loss Functions**: Various for diff. tasks:
  - `torch.nn.MSELoss`: Mean Squared Error (MSE) for regression tasks.
  - `torch.nn.BCELoss`: Binary Cross Entropy Loss, binary classification.
  - `torch.nn.CrossEntropyLoss`: Cross-Entropy Loss, multi-class classifn.
- **Optimizers**: (update weights during training)
  - Stochastic Gradient Descent (SGD): `torch.optim.SGD`.
  - Adam: `torch.optim.Adam`.
  - **Important optimizer functions**:
    `optimizer.zero_grad()`: Resets gradients to zero before computing new gradients.
    `optimizer.step()`: Updates the weights after backpropagation.

Building a Neural Network Example



Training a Neural Network Example